

Curso online: **Programación orientada a objetos: Java avanzado**

Sockets

Autores

Jorge Molinero Muñoz

Miguel Sallent Sánchez

ÍNDICE

1.	Introducción.....	3
2.	Sockets en Java.....	4
3.	Conexiones a URLs.....	10
3.1.	La clase URL.....	10
3.2.	Las clases URLConnection y HttpURLConnection.....	12
4.	Ejercicio resuelto.....	13

1. Introducción

Hoy en día es habitual que las aplicaciones se conecten entre sí para intercambiar datos. El caso más habitual es el de dos aplicaciones que se encuentran en dos ordenadores distintos conectados a través de Internet. Los sockets nos permiten establecer un canal de comunicación entre los dos ordenadores.

En las redes IP como Internet hay dos tipos de comunicaciones:

- De flujo: es el caso más habitual. Son comunicaciones orientadas a conexión: primero se establece la conexión y luego se establece un flujo de datos ordenado y seguro. El protocolo que implementa este tipo de flujo es TCP. Por ejemplo, se puede utilizar para descargar una página web.
- De datagrama: es un flujo sin conexión, se envían los datos en pequeñas unidades, denominadas paquetes o datagramas, y no se garantiza ni un orden de llegada ni tampoco que lleguen todos los paquetes. Se utiliza por ejemplo para flujos de audio o vídeo, en el que la pérdida de algún paquete no supone un problema y es preferible descartarlo a solicitar su retransmisión, para evitar retardos. El protocolo que implementa este tipo de flujo es UDP.

El protocolo más utilizado es TCP, y por ello será el que se explicará en este curso.

En la arquitectura TCP/IP cada una de las dos aplicaciones adopta un rol:

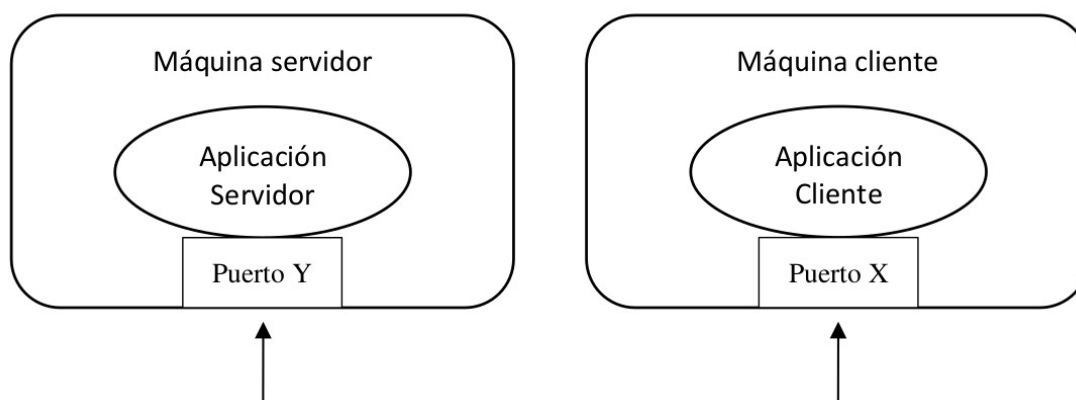
- Aplicación servidor: es la aplicación que atiende peticiones de los clientes, por tanto se mantiene “a la escucha”, a la espera de que un cliente solicite abrir la comunicación.
- Aplicación cliente: es la aplicación que realiza peticiones de comunicación a los servidores. Por tanto inicia la comunicación, enviando una petición de apertura de comunicación al servidor. Si este la acepta se establecerá la comunicación, y una vez establecida se podrá iniciar el intercambio bidireccional de datos.

El protocolo TCP permite que una máquina tenga múltiples aplicaciones clientes y servidores, todos trabajando a la vez. Para permitir esto, define el concepto de “puerto”, de forma que un servidor puede tener una aplicación escuchando en un puerto y otra aplicación al mismo tiempo escuchando en otro puerto. El cliente solicitará abrir conexión con un puerto u otro según la aplicación con la que se quiera conectar. Por ejemplo, si se quiere conectar al servidor web para descargar una página web, se conectará al puerto 80, y si se quiere conectar al servidor de correo para enviar un email se conectará al puerto 25.

Al conjunto de dirección IP + puerto se le denomina **socket**. Es decir, si una máquina tiene la dirección IP 192.168.1.13 y tiene una aplicación escuchando en el puerto 8080, el socket será 192.168.1.13:8080 (se suele representar separando la dirección y el puerto por dos puntos). Los puertos pueden tomar cualquier valor entre 0 y 65535.

Del mismo modo, en el lado del cliente también se utilizará un puerto distinto para cada conexión, de esta forma cuando el servidor envíe datos al cliente, indicará el puerto del cliente al que envía los datos y así el cliente podrá saber a qué aplicación van dirigidos.

Por tanto un canal de comunicación queda definido por dos sockets: el socket cliente y el socket servidor. El puerto en el lado cliente y el puerto en el lado servidor no tienen por qué coincidir, de hecho suelen ser distintos.



En resumen:

- Un socket es un extremo de una comunicación bidireccional entre dos aplicaciones.
- Un socket queda definido por la dirección IP de la máquina y un puerto.
- Hay dos tipos de sockets: clientes y servidores.

A continuación veremos cómo crear y utilizar sockets TCP en Java, tanto clientes como servidores.

2. Sockets en Java

En el siguiente ejemplo vamos a escribir dos programas, uno cliente y otro servidor. Ambos se ejecutarán en la misma máquina, pero podrían ejecutarse en máquinas distintas. El cliente abrirá una consola y pedirá al usuario que introduzca un número entero. Al pulsar la tecla ENTER se enviará el número indicado al servidor. El servidor devolverá dicho número multiplicado por dos, y el cliente lo mostrará por pantalla. Cuando el cliente introduzca el comando “exit”, se desconectará. Vemos que ambas aplicaciones tendrán que leer y escribir datos en el socket.

Comencemos por el cliente, el código sería el siguiente:

```

package tema1;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.Scanner;

public class MultiplyClient {

    public static void main(String[] args) {

        String serverIP = "127.0.0.1";
        int serverPort = 12345;

        try (
            //abrimos el socket:
            Socket socket = new Socket(serverIP, serverPort);

            //stream de salida
  
```

```

PrintWriter out =
    new PrintWriter(socket.getOutputStream(), true);

//stream de entrada
BufferedReader in =
    new BufferedReader(
        new InputStreamReader(socket.getInputStream()));

//leemos la entrada del usuario:
Scanner userIn = new Scanner(System.in);

) {
    boolean terminar = false;

    while (!terminar) {
        System.out.print("Introduzca un número (o exit para terminar): ");

        String userInput = userIn.nextLine();

        if ("exit".equals(userInput)) {
            terminar = true;
        } else {
            //enviamos el número al servidor:
            out.println(userInput);
            //leemos el mensaje del servidor y lo mostramos:
            System.out.println("Respuesta del servidor: " + in.readLine());
        }
    }
    System.out.println("Programa finalizado.");
} catch (IOException e) {
    System.out.println(
        "Se ha producido un error: " + e.getMessage());
}
}
}

```

Explicamos detalladamente cada paso:

En primer lugar se abre el Socket. Como se ha indicado en la introducción es necesario indicar la dirección IP y el puerto del servidor destino:

```
Socket socket = new Socket(serverIP, serverPort);
```

La dirección IP 127.0.0.1 corresponde a la máquina en que se ejecuta el programa (localhost). No es necesario indicar el puerto origen, el sistema asignará uno cualquiera que esté disponible. Esto evita tener que buscar un puerto libre.

El objeto Socket nos proporciona dos streams para acceder a los datos, uno de entrada y otro de salida, mediante los métodos `getInputStream` y `getOutputStream`. El stream de entrada servirá para leer datos que nos envía el servidor. El stream de salida servirá para escribir y enviar datos al servidor. Ambos streams son de bytes, pero puesto que los mensajes que vamos a intercambiar son de texto, los conectamos a un `PrintWriter` y un `BufferedReader` respectivamente, ya que estos objetos realizan la transformación de bytes a texto y viceversa.

A continuación creamos un objeto Scanner que nos permite leer los mensajes que el usuario escribe por consola.

Estos cuatro objetos (Socket, Reader, Writer y Scanner) los creamos dentro de lo que se llama un **try-with-resources**:

```
try(resources) {
    ...
} catch (Exception e) {
    ...
}
```

Los recursos creados dentro del bloque entre paréntesis son cerrados al terminar el bloque try, lo que es equivalente a incluir una cláusula finally cerrando todos los recursos. Para ello, los recursos deben implementar el interfaz AutoCloseable, de modo que se invocará el método close() al finalizar el bloque finally. Este tipo de sentencias, simplifica en gran medida el código ya que evita estar pendiente de cerrar dichos elementos al finalizar el bloque try.

Dentro del bloque try abrimos un bucle. Pedimos una orden al usuario usando el objeto Scanner, y posteriormente se la enviamos al servidor usando el Writer. Para leer la respuesta del servidor usaremos el Reader. Mantendremos el bucle en ejecución hasta que el usuario introduzca la orden "exit".

A continuación escribiremos el código del servidor:

```
package tema1;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;

public class MultiplyServer {

    public static void main(String[] args) {

        int portNumber = 12345;
        System.out.println("Servidor en ejecución, esperando conexión...");

        try (ServerSocket serverSocket = new ServerSocket(portNumber);
            Socket clientSocket = serverSocket.accept();
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new
            InputStreamReader(clientSocket.getInputStream()));) {

            System.out.println("Se ha conectado un cliente");

            String inputLine;
            while ((inputLine = in.readLine()) != null) {
                System.out.println("Se ha recibido una orden: -> " + inputLine);
                try {
                    long received = Long.parseLong(inputLine);
                    out.println(received*2);
                } catch (NumberFormatException e) {
                    out.println("El texto recibido no es un número válido");
                }
            }
        }
    }
}
```

```

    }
    System.out.println("Se ha desconectado el cliente");
    System.out.println("Programa finalizado");
} catch (IOException e) {
    System.out.println(
        "Se ha producido un error: " + e.getMessage());
}
}
}
}

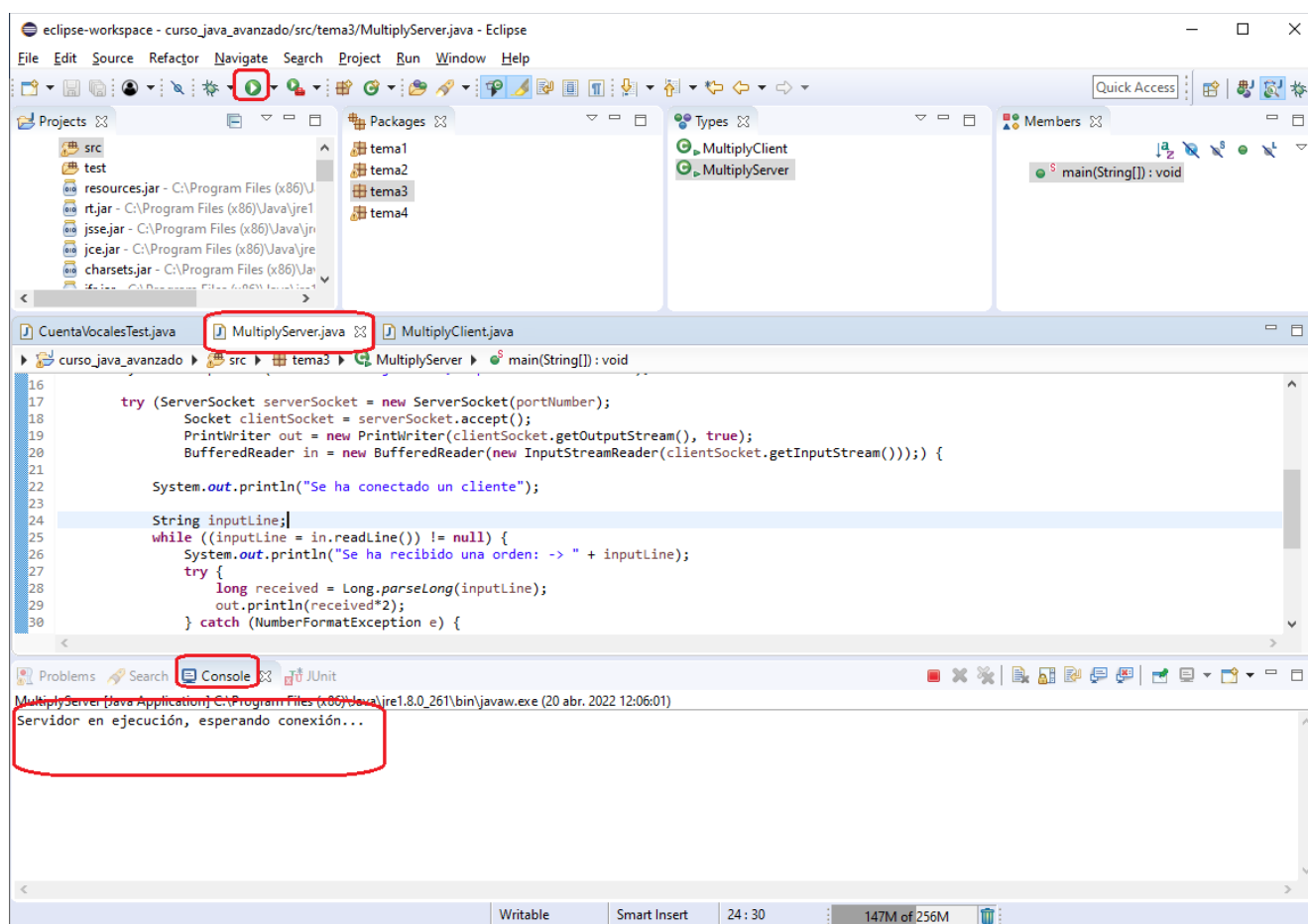
```

El código es similar al del cliente. Sin embargo, para abrir el socket utilizamos la siguiente instrucción:

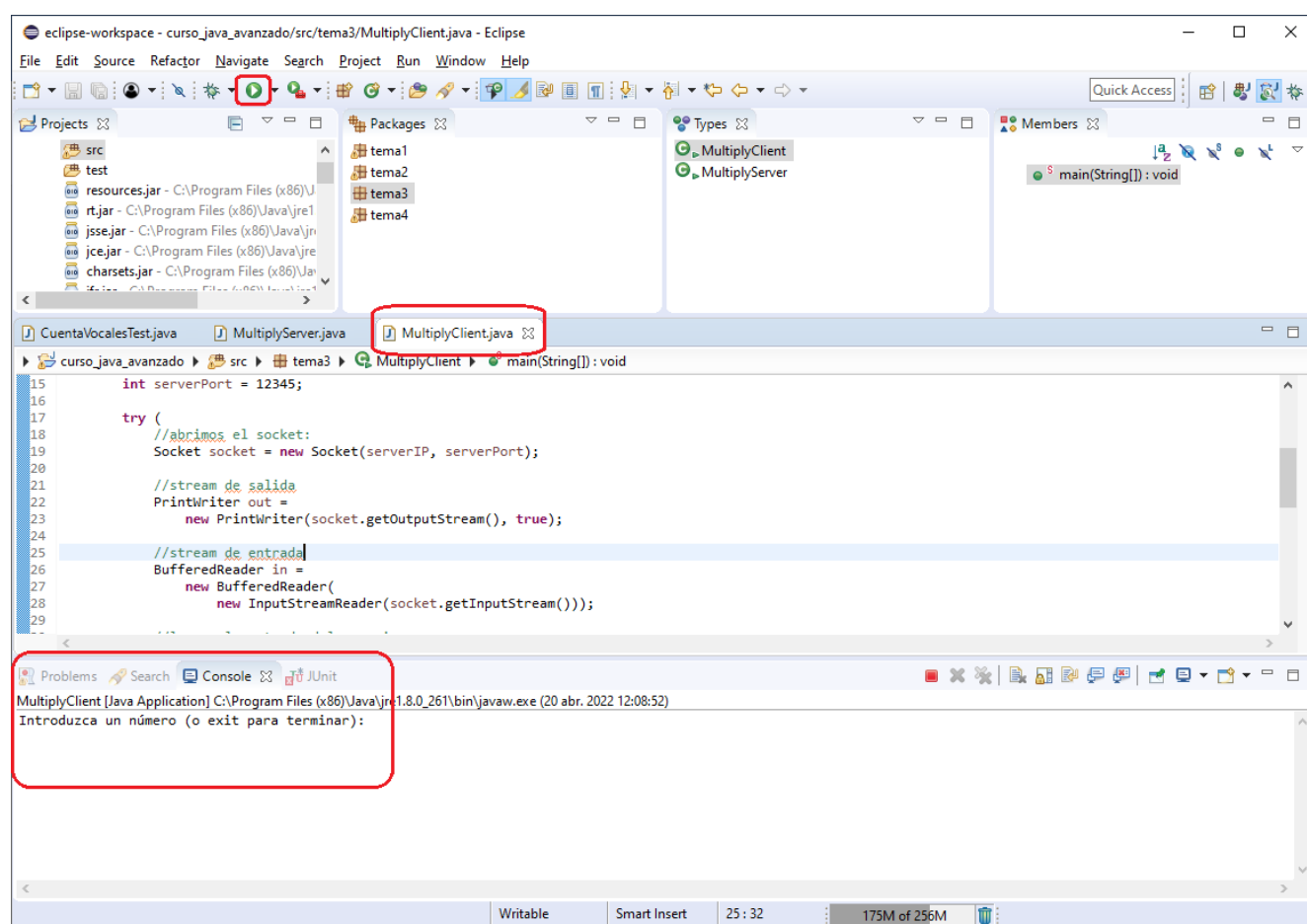
```
Socket clientSocket = serverSocket.accept();
```

El método accept de la clase ServerSocket abre un socket de escucha. Este método bloquea la ejecución del código, que se queda esperando hasta que se conecta un cliente. Cuando el cliente se conecta, el método accept devuelve un objeto Socket que ya es igual que el del cliente, y nos permite acceder a los streams de entrada y salida para intercambiar datos con el cliente. Al igual que con el cliente utilizamos un bloque try-with-resources para ahorrarnos incluir la cláusula finally. En este caso no hay objeto Scanner porque el servidor no interactúa con el usuario, únicamente interactúa con el cliente.

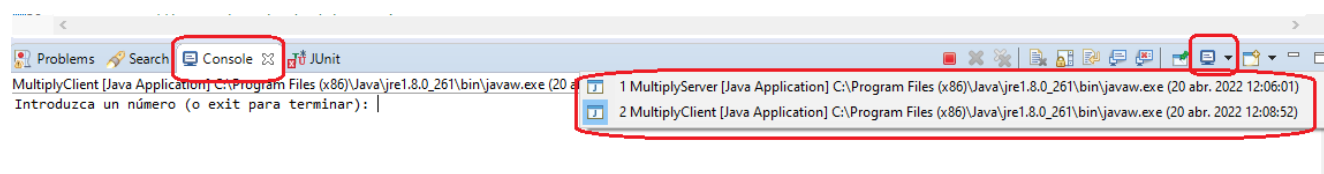
Veamos ahora cómo ejecutar el cliente y el servidor en Eclipse. Téngase en cuenta que vamos a tener dos aplicaciones funcionando a la vez. Para ello primero nos situaremos en la clase MultiplyServer y pulsaremos el botón de iniciar. En la consola inferior veremos el mensaje:



A continuación haremos lo mismo para el cliente:

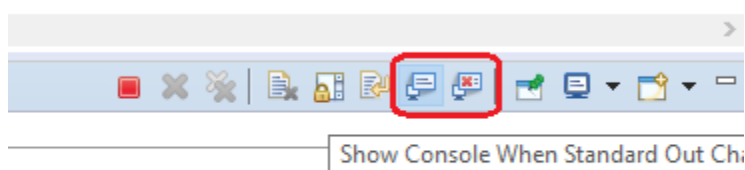


Puesto que tenemos dos aplicaciones en ejecución, tenemos dos consolas. Podemos alternar entre una y otra mediante el desplegable a la derecha:

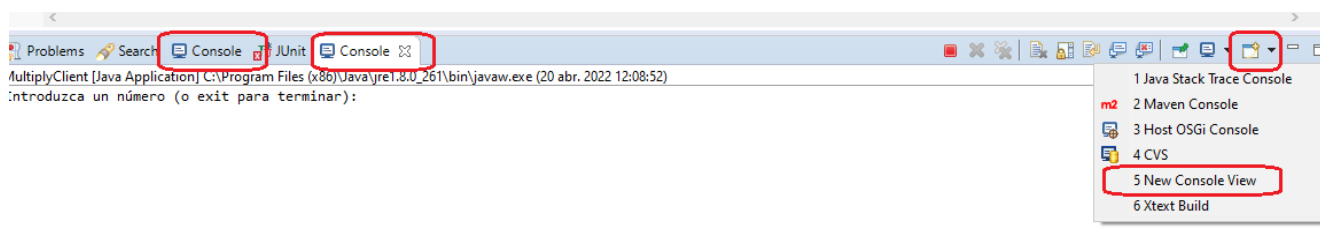


También es posible abrir dos ventanas, una para cada consola. Para ello seguimos los siguientes pasos:

- Asegurarse de tener desmarcados los botones "Show Console When Standard Out Changes" y "Show Console When Standard Error Changes". Esto evita que cambie de consola automáticamente.



- En el menú de la derecha seleccionamos New Console View. Nos aparecerá otra ventana de consola:



- Una vez arrancadas ambas aplicaciones, en una de las consolas seleccionamos el cliente y en la otra el servidor, mediante el desplegable mostrado anteriormente.

Tras hacer unas pruebas insertando datos en el cliente, las consolas del cliente y el servidor se mostrarían así:

```

Problems Search Console JUnit Console
MultiplyClient [Java Application] C:\Program Files (x86)\Java\jre1.8.0_261\bin\javaw.exe (20 abr. 2022 12:08:52)
Introduzca un número (o exit para terminar):
<terminated> MultiplyClient [Java Application] C:\Program Files (x86)\Java\jre1.8.0_261\bin
Introduzca un número (o exit para terminar): 3
Respuesta del servidor: 6
Introduzca un número (o exit para terminar): 125
Respuesta del servidor: 250
Introduzca un número (o exit para terminar): 15432
Respuesta del servidor: 30864
Introduzca un número (o exit para terminar): hola
Respuesta del servidor: El texto recibido no es un número válido
Introduzca un número (o exit para terminar): exit
Programa finalizado.

```

```

Problems Search Console JUnit Console
MultiplyServer [Java Application] C:\Program Files (x86)\Java\jre
Servidor en ejecución, esperando conexión...
Se ha conectado un cliente
Se ha recibido una orden: -> 3
Se ha recibido una orden: -> 125
Se ha recibido una orden: -> 15432
Se ha recibido una orden: -> hola
Se ha desconectado el cliente
Programa finalizado

```

Este servidor es muy sencillo, cuando el cliente se desconecta el servidor finaliza. Se podría modificar para que funcionara en bucle, de forma que cuando se desconecta un cliente el servidor volviera de nuevo a “escuchar” quedando a la espera de un nuevo cliente.

Se propone al alumno que realice las siguientes pruebas:

- Arrancar el servidor. Con un servidor todavía en ejecución, intentar arrancar otro servidor. Al intentar arrancar el segundo deberá dar un error, ya que no es posible escuchar en un puerto si ya hay otra aplicación escuchando en ese puerto.

- Intentar arrancar el cliente sin que el servidor esté arrancado. Deberá dar un error indicando que no se puede conectar.

3. Conexiones a URLs

Hasta ahora hemos visto como abrir conexiones TCP utilizando sockets, que permiten el intercambio de cualquier tipo de datos. El cliente y el servidor que utilizan un canal TCP deben ponerse de acuerdo en el tipo de mensajes que van a intercambiar, su significado, formato, orden, etc. Esto es lo que se conoce como “protocolo”.

Hay muchos protocolos estandarizados, como por ejemplo HTTP para descargar páginas web, SMTP para envío de correos electrónicos, FTP para transferencia de archivos, etc. Java proporciona implementaciones de clientes para muchos de ellos, de forma que el programador no tiene que trabajar con sockets e implementar el protocolo.

Por tanto, por regla general sólo tendremos que utilizar los sockets cuando queramos implementar nuestro propio protocolo, y en el resto de casos utilizaremos las clases que Java proporciona y que veremos a continuación.

3.1. La clase URL

Una URL es una cadena de texto que nos permite definir el conjunto protocolo + socket, es decir, protocolo + máquina + puerto. Adicionalmente se puede indicar un recurso a obtener. Por ejemplo:

http://www.example.com:8080/index.php?userid=test
 { { { { {
 protocol host port path query

El significado de cada elemento es el siguiente:

- Protocol: indica el protocolo. Por ejemplo http, https, ftp, etc.
- Host: nombre o dirección IP de la máquina.
- Port: puerto TCP. Es opcional, si no se indica se utilizará el puerto por defecto para el protocolo indicado.
- Path: recurso a obtener. También es opcional, si no se indica se tomará el recurso por defecto.
- Query: información adicional para el recurso. Opcional.

En Java disponemos de la clase [java.net.URL](#), que nos permite trabajar con URLs facilitándonos su procesamiento. Por ejemplo:

```
package tema1;

import java.net.URL;

public class UrlExample1 {

    public static void main(String[] args) throws Exception {

        URL url=new URL("http://www.example.com:8080/index.php?user=test&password=test");

        System.out.println("Protocol: "+ url.getProtocol());
    }
}
```

```

System.out.println("Host: "+ url.getHost());
System.out.println("Port: "+url.getPort());
System.out.println("Path: "+ url.getPath());
System.out.println("Query String: "+ url.getQuery());
System.out.println("File: "+ url.getFile());

}

}

```

La salida será la siguiente:

```

Protocol: http
Host: www.example.com
Port: 8080
Path: /index.php
Query String: user=test&password=test
File: /index.php?user=test&password=test

```

La clase URL proporciona un método [openStream\(\)](#) que nos permite acceder directamente a un Stream con los datos de la URL. Por ejemplo:

```

package tema1;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.URL;

public class UrlExample2 {

    public static void main(String[] args) throws Exception {

        URL url = new URL("https://java.com");

        try
        (
            BufferedReader in = new BufferedReader(
                new InputStreamReader(url.openStream()));
        ){
            String inputLine;
            while ((inputLine = in.readLine()) != null) {
                System.out.println(inputLine);
            }
        }
        catch (Exception e) {
            System.out.println("Ocurrió un error: " + e.getMessage());
        }
    }
}

```

Si ejecutamos el código anterior veremos por pantalla el código HTML de la página web descargada. En este ejemplo no se ha indicado el puerto en la URL, en ese caso utilizará el puerto por defecto de HTTP que es 80.

3.2. Las clases `URLConnection` y `HttpURLConnection`

El método `openStream` únicamente nos permite obtener un flujo de entrada, para tener más control sobre la conexión es necesario obtener un objeto [URLConnection](#) a través del método `openConnection` de la clase `URL`.

La clase [URLConnection](#) es abstracta, en función del protocolo utilizado el objeto será de una subclase o de otra. Así, si el protocolo indicado en la url es `http` o `https`, el objeto devuelto será [HttpURLConnection](#). Deberemos hacer un cast para acceder a los métodos específicos de esta clase, que nos permitirán acceder a los datos del protocolo HTTP, como pueden ser las cabeceras, el status code de respuesta, etc. También podemos modificar el método `http` por ejemplo a `POST` y enviar datos utilizando el stream devuelto por el método `getOutputStream()`.

En el siguiente ejemplo haremos una petición a un servidor HTTP. No sólo mostraremos el resultado devuelto por el servidor, sino también todas las cabeceras HTTP recibidas, así como el status code.

```
package tema1;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.List;
import java.util.Map;

public class HttpExample {

    public static void main(String[] args) throws Exception {

        URL url = new URL("https://java.com");
        HttpURLConnection con = (HttpURLConnection) url.openConnection();

        int responseCode = con.getResponseCode();
        System.out.println("Response Code: " + responseCode);

        System.out.println("=====");
        System.out.println("Response Headers:");
        System.out.println();
        Map<String, List<String>> responseHeaders = con.getHeaderFields();
        responseHeaders.entrySet().stream().forEach(
            e -> System.out.println(e.getKey() + ": " +
                e.getValue().stream().findFirst().get()));

        try(
            BufferedReader in = new BufferedReader(new InputStreamReader(
                con.getInputStream()))
        ){
            String inputLine;
            StringBuffer response = new StringBuffer();

            while ((inputLine = in.readLine()) != null) {
                response.append(inputLine);
            }
            System.out.println("=====");
            System.out.println("Response:");
            System.out.println();
            System.out.println(response.toString());
        }
    }
}
```

```
}
}
```

4. Ejercicio resuelto

En el siguiente ejercicio veremos un ejemplo de lo que se denomina “web scraping”: desarrollar un programa que se conecta a una página web para recopilar información de forma automática. Un ejemplo son los rastreadores de los buscadores tipo Google. A este tipo de programas se les denomina “robots” o simplemente “bots”.

El alumno deberá desarrollar una aplicación que se conecte a una página web y consulte todos los enlaces que contiene, mostrando una estadística de cuántos enlaces hay por cada host distinto.

Los enlaces son elementos xml, en el que el nombre es “a” y el enlace está en el atributo “href”. Por ejemplo:

```
<a href="http://www.example.com">Este es un ejemplo de enlace</a>
```

Ojo, los enlaces en ocasiones están en varias líneas. Se recomienda sustituir todos los retornos de carro por espacios en blanco.

```
<a href="http://www.example.com">
  Este es un enlace en varias líneas.
</a>
```

No se recomienda el tratamiento del HTML como un XML, ya que algunas páginas web no tienen un XML bien formado, lo que impide el parseado. Es mejor realizarlo con búsquedas de texto o [expresiones regulares](#).

Una vez tengamos la lista de elementos href enlazados, hay que tener en cuenta lo siguiente:

1. Todos los enlaces que no empiecen por “http://”, “https://”, “./”, “../” o “/” deben ignorarse. Ejemplos de enlaces que deben ignorarse:
 - href=”javascript:void(0)”
 - href=”mailto:test@test.com”
 - href=”#top”
2. Los enlaces que empiezan por /, ./ o ../ son enlaces al mismo site que se está comprobando. Ejemplos:


```
href="/index.html"
href="./index.html"
href="../index.html"
```

Una vez tengamos la lista de elementos href que corresponden a URLs válidas, se extrae el host de cada URL y se calculan las estadísticas.

Ejemplo del resultado de la aplicación:

```
Nº de enlaces por host para el site https://www.xataka.com/
www.xataka.com -> 269/444 (60,59%)
www.vidaextra.com -> 10/444 (2,25%)
magnet.xataka.com -> 10/444 (2,25%)
www.3djuegos.com -> 9/444 (2,03%)
www.xatakamovil.com -> 8/444 (1,80%)
www.youtube.com -> 6/444 (1,35%)
www.applesfera.com -> 6/444 (1,35%)
```

```

www.xatakandroid.com -> 6/444 (1,35%)
www.genbeta.com -> 5/444 (1,13%)
www.espinof.com -> 4/444 (0,90%)
instagram.com -> 4/444 (0,90%)
www.compradiccion.com -> 4/444 (0,90%)
www.trendencias.com -> 4/444 (0,90%)
www.xatakahome.com -> 4/444 (0,90%)
www.trendenciashombre.com -> 4/444 (0,90%)
www.xatakawindows.com -> 4/444 (0,90%)
www.poprosa.com -> 4/444 (0,90%)
www.sensacine.com -> 4/444 (0,90%)
www.mundoxiaomi.com -> 4/444 (0,90%)
www.tiktok.com -> 4/444 (0,90%)
twitter.com -> 4/444 (0,90%)
telegram.me -> 4/444 (0,90%)
www.3djuegosguias.com -> 4/444 (0,90%)
www.xatakafoto.com -> 4/444 (0,90%)
www.vitonica.com -> 4/444 (0,90%)
www.3djuegospc.com -> 4/444 (0,90%)
www.directoalpaladar.com -> 4/444 (0,90%)
decoracion.trendencias.com -> 4/444 (0,90%)
www.xatakaciencia.com -> 4/444 (0,90%)
youtu.be -> 3/444 (0,68%)
www.facebook.com -> 3/444 (0,68%)
www.linkedin.com -> 3/444 (0,68%)
flipboard.com -> 3/444 (0,68%)
www.weblogssl.com -> 2/444 (0,45%)
www.sensacine.com.mx -> 2/444 (0,45%)
innovacionvolvo.xataka.com -> 2/444 (0,45%)
www.webedia.es -> 2/444 (0,45%)
www.3djuegos.lat -> 2/444 (0,45%)
www.directoalpaladar.com.mx -> 2/444 (0,45%)
www.xataka.com.mx -> 2/444 (0,45%)
fullelectric.xataka.com -> 2/444 (0,45%)
tecnologiaalg.xataka.com -> 2/444 (0,45%)
www.getrevue.co -> 1/444 (0,23%)
www.twitch.tv -> 1/444 (0,23%)
premios.xataka.com -> 1/444 (0,23%)
weblogs.webedia.es -> 1/444 (0,23%)
bit.ly -> 1/444 (0,23%)

```

NOTA: puesto que la página web del ejemplo puede ser actualizada en cualquier momento, el alumno puede obtener un resultado que no sea exactamente igual al mostrado.

La solución al ejercicio está disponible en la plataforma del INAP.